

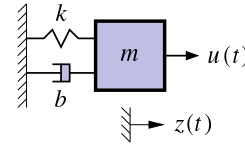


The example we study this week

- The system we will use as an example of a KF implementation is the spring-mass-damper system introduced in week 2.
- We found the continuous-time state-space model to be:

$$\dot{x}(t) = \underbrace{\begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{b}{m} \end{bmatrix}}_A x(t) + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}_B u(t) + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}_{B_w} w(t).$$

$$z(t) = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_C x(t) + \underbrace{\begin{bmatrix} 0 \end{bmatrix}}_D u(t) + v(t).$$



- To flesh out this model,
 - We must define b , k , and m ;
 - We must describe the noises $w(t)$ and $v(t)$; and
 - We must specify a sample period Δt and convert dynamics to discrete-time.



Putting in numbers

- For the purpose of example, let $m = 50$, $b = 2$, and $k = 4$.
- Substituting, the continuous-time state-space model is:

$$\dot{x}(t) = \begin{bmatrix} 0 & 1 \\ -0.08 & -0.04 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 0.02 \end{bmatrix} u(t) + \begin{bmatrix} 0 \\ 0.02 \end{bmatrix} w(t)$$

$$z(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} x(t) + \begin{bmatrix} 0 \end{bmatrix} u(t) + v(t).$$

- Further, we let both $w(t)$ and $v(t)$ be zero-mean white Gaussian processes having covariances $S_w = 0.01$ and $S_v = 0.00001$.
- Let the sampling period $\Delta t = 0.1$ s.



Converting to discrete-time

- Following “a little trick” in Lesson 1.3.7, define:

$$Z = \begin{bmatrix} -A & B_w S_w B_w^T \\ 0 & A^T \end{bmatrix}, \quad C = e^{Z \Delta t} = \begin{bmatrix} c_{11} & c_{12} \\ 0 & c_{22} \end{bmatrix}.$$

- Then, recall: $A_d = c_{22}^T$, $B_d = A^{-1}(A_d - I)B$, $\Sigma_{\tilde{w}} = c_{22}^T c_{12}$, and $\Sigma_{\tilde{v}} = S_v / \Delta t$.
- The discrete-time state-space model is:

$$x_{k+1} = \begin{bmatrix} 0.9996 & 0.09979 \\ -0.007983 & 0.9956 \end{bmatrix} x_k + \begin{bmatrix} 9.986 \times 10^{-5} \\ 0.001996 \end{bmatrix} u_k + w_k$$

$$z_k = \begin{bmatrix} 1 & 0 \end{bmatrix} x_k + v_k.$$

- Also, $\Sigma_{\tilde{w}} = \begin{bmatrix} 0.0013 & 0.0199 \\ 0.0199 & 0.3983 \end{bmatrix} \times 10^{-6}$ and $\Sigma_{\tilde{v}} = 0.0001$.



Simulating the model in Octave: Setup

- To simulate the model in Octave, we first set up the problem.

```
m = 50; b = 2; k = 4; % Specify physical constants
A = [0 1; -k/m -b/m]; % Specify continuous-time model
B = [0; 1/m];
C = [1 0]; D = 0;

dT = 0.1; % Create discrete-time model
Ad = expm(A*dT);
Bd = inv(A)*(Ad-eye(2))*B;
Cd = C; Dd = D;

Sw = 0.01; Sv = 1e-5; % Specify continuous-time noise model parameters
Bw = B;
Z = [-A Bw*Sw*Bw'; ... % Convert to discrete-time using "a little trick"
     zeros(2) A'];
C = expm(Z*dT);
c12 = C(1:2,3:4);
c22 = C(3:4,3:4);
SigmaW = c22'*c12; % Process-noise covariance matrix
SigmaV = Sv/dT; % Sensor-noise covariance matrix
```



Simulating the model in Octave: Execution

- With all the variables defined, we can simulate the model.

```
t = 0:dT:100; % Define a simulation time vector
u = sin(2*pi*(1/10)*t); % Define an interesting input signal

x = zeros(2,length(t)+1); % Initialize variables to store state...
z = zeros(1,length(t)); % ... and measurement

for k = 1:length(t) % Enter the simulation loop
    x(:,k+1) = Ad*x(:,k) + Bd*u(k) + chol(SigmaW,'lower')*randn(2,1);
    z(k) = Cd*x(:,k) + Dd*u(k) + chol(SigmaV,'lower')*randn(1);
end
x = x(:,1:length(t)); % Crop states to same length as t

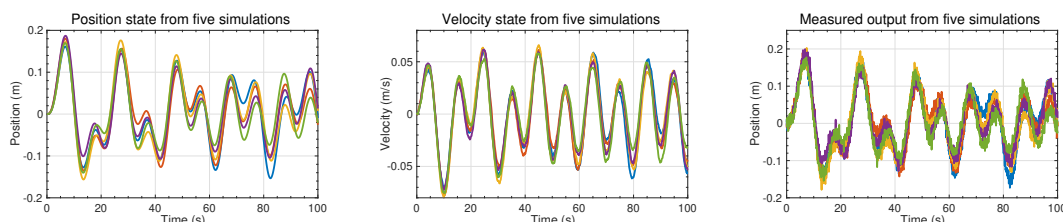
figure(1); plot(t,x(1,:)); % Plot the position state
figure(2); plot(t,x(2,:)); % Plot the velocity state
figure(3); plot(t,z); % Plot the (noisy) position measurement

% Save parameters and signals to a file for later use by the KF
save simOut.mat Ad Bd Cd Dd SigmaW SigmaV dT t u x z
```



Sample output

- The figures below show sample output for five simulations using different random w_k and v_k signals.



- The sinusoidal input causes the position to be nearly sinusoidal—if there were no process noise, the block position would settle to a steady-state sinusoidal signal.
 - That is, the process noise causes the trajectory to be different from a pure sinusoid, perturbing both position and velocity.
- The measurement noise causes the measured and true positions to differ.



Summary

- You have now seen a complete example showing how to prepare a model for use by the KF:
 - We defined a continuous-time state-space model of a spring-mass-damper;
 - We then converted these dynamics to a discrete-time state-space form;
 - We also converted the noise-model covariance matrices to their equivalent discrete-time versions.
- You learned how to simulate this model in Octave code, and saw some representative outputs from the simulations.
- We have now saved results from a simulation to `simOut.mat`, to be loaded by the KF code and to be used to demonstrate a KF in operation.
- We're now ready to implement a KF in Octave and to look at how it performs!